

Universidad Mariano Gálvez de Guatemala
Campus Cobán
Ingeniería en Sistemas y Ciencias de la Computación
Programación I



Sistema de Gestión de Inventario
"Almacén El Dorado"

Integrantes:

Fredy Rudy Willson Humbler Cuc

Endso Felipe Ramírez Teec

Carnés:

0902 25 12915

0902 25 22181

Coban, 2026

1. INTRODUCCIÓN

El presente proyecto tiene como finalidad integrar los conocimientos adquiridos durante el curso de Programación I, mediante el desarrollo de una aplicación completa utilizando el lenguaje C#.

El proyecto se enfoca en la aplicación de los principios de la programación orientada a objetos, así como en la interacción con bases de datos para la persistencia de la información.

El sistema desarrollado es un **Sistema de Gestión de Inventario** para pequeñas tiendas o bodegas, que permite administrar productos, controlar existencias y registrar movimientos de entrada y salida.

El sistema está compuesto por:

- Backend con lógica de negocio en C#
- Base de datos SQL Server para almacenamiento de información
- Interfaz de usuario WinForms para interacción

2. PROBLEMA ABORDADO

Muchas pequeñas empresas no cuentan con un sistema automatizado para gestionar su inventario, lo que provoca:

- Falta de control sobre los productos disponibles
- Errores en el registro de entradas y salidas
- Pérdidas por desabastecimiento o sobrestock
- Dificultad para consultar información en tiempo real

Ante esta problemática, se propone el desarrollo de un sistema informático que permita llevar un control preciso y actualizado del inventario, facilitando la toma de decisiones y mejorando la organización.

El sistema permite:

- Registrar información de productos, categorías y proveedores
- Gestionar datos mediante operaciones CRUD
- Consultar información de forma eficiente

3. OBJETIVOS

3.1 Objetivo General

Desarrollar una aplicación en C# que implemente principios de programación orientada a objetos e integración con una base de datos SQL Server para la gestión de inventario.

3.2 Objetivos Específicos

- Aplicar conceptos de clases, objetos, herencia y encapsulamiento
- Diseñar un modelo de datos coherente con entidades y relaciones
- Implementar operaciones CRUD (Create, Read, Update, Delete)
- Conectar una aplicación C# con una base de datos SQL Server
- Desarrollar una interfaz de usuario funcional en WinForms
- Utilizar control de versiones con GitHub

4. ARQUITECTURA DEL SISTEMA

El sistema implementa una arquitectura de tres capas:

4.1 Capa de Presentación (Frontend)

Responsable de la interacción con el usuario, visualización de datos y captura de información.

Formularios desarrollados:

- FrmLogin: Autenticación de usuarios
- FrmPrincipal: Menú principal
- FrmProductos: Gestión de productos
- FrmCategorias: Gestión de categorías
- FrmProveedores: Gestión de proveedores
- FrmMovimientos: Entradas y salidas de inventario
- FrmConsultas: Consulta de inventario

Tecnología: Windows Forms (.NET Framework)

4.2 Capa de Lógica de Negocio (Backend)

Responsable de las reglas del sistema, procesamiento de información y validaciones.

Validaciones implementadas:

- No permitir salidas si no hay stock suficiente
- Validar campos obligatorios (código, nombre, precio)
- Validar que el código de producto sea único
- Validar que los precios y cantidades sean números positivos

Tecnología: C#

4.3 Capa de Datos

Responsable de la persistencia de información y consultas a la base de datos.

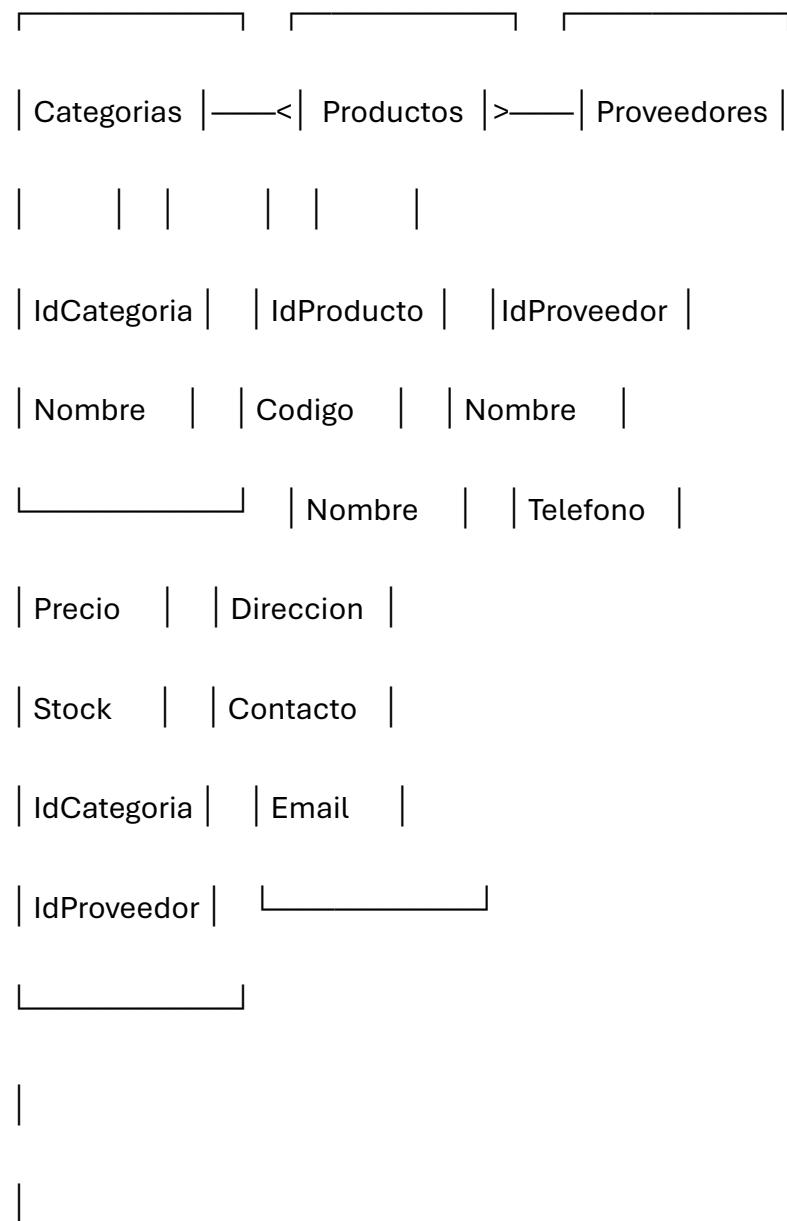
Componentes:

- DatabaseHelper: Cadena de conexión
- Conexion: Manejo de conexiones SQL

Tecnología: SQL Server con [ADO.NET](#)

5. MODELO DE DATOS

5.1 Diagrama de Entidades



| Movimientos |

| |

| IdMovimiento |

| Tipo |

| Cantidad |

| Fecha |

| IdProducto |

5.2 Tablas y Descripciones

Tabla: Categorías

Campo	Tipo	Descripción
IdCategoria	INT (PK, IDENTITY)	Identificador único de la categoría
Nombre	VARCHAR(100)	Nombre de la categoría

Tabla: Proveedores

Campo	Tipo	Descripción
IdProveedor	INT (PK, IDENTITY)	Identificador único del proveedor
Nombre	VARCHAR(100)	Nombre del proveedor
Telefono	VARCHAR(20)	Teléfono de contacto

Campo	Tipo	Descripción
Direccion	VARCHAR(200)	Dirección física
Contacto	VARCHAR(100)	Persona de contacto
Email	VARCHAR(100)	Correo electrónico

Tabla: Productos

Campo	Tipo	Descripción
IdProducto	INT (PK, IDENTITY)	Identificador único del producto
Codigo	VARCHAR(50)	Código único del producto
Nombre	VARCHAR(100)	Nombre del producto
Precio	DECIMAL(10,2)	Precio del producto
Stock	INT	Cantidad en inventario
IdCategoria	INT (FK)	Referencia a Categorías

Campo	Tipo	Descripción
-------	------	-------------

IdProveedor	INT (FK)	Referencia a Proveedores
-------------	----------	--------------------------

Tabla: Movimientos

Campo	Tipo	Descripción
-------	------	-------------

IdMovimiento	INT (PK, IDENTITY)	Identificador del movimiento
--------------	--------------------	------------------------------

Tipo	VARCHAR(10)	Tipo: 'ENTRADA' o 'SALIDA'
------	-------------	----------------------------

Cantidad	INT	Cantidad del movimiento
----------	-----	-------------------------

Fecha	DATETIME	Fecha y hora del movimiento
-------	----------	-----------------------------

IdProducto	INT (FK)	Referencia a Productos
------------	----------	------------------------

5.3 Relaciones entre Tablas

Relación	Tipo	Explicación
----------	------	-------------

Productos → Categorías	N:1	Un producto pertenece a una categoría
--	----------------	--

Relación	Tipo	Explicación
Productos → Proveedores	N:1	Un producto puede tener un proveedor
Movimientos → Productos	N:1	Un movimiento pertenece a un producto

5.4 Script de Creación de Base de Datos

```

sql
CREATE DATABASE AlmacenElDorado;
GO

USE AlmacenElDorado;
GO

-- Tabla de Categorías
CREATE TABLE Categorías (
  IdCategoría INT PRIMARY KEY IDENTITY(1,1),
  Nombre VARCHAR(100) NOT NULL
);
GO

-- Tabla de Proveedores

CREATE TABLE Proveedores (

  IdProveedor INT PRIMARY KEY IDENTITY(1,1),

  Nombre VARCHAR(100) NOT NULL,

  Telefono VARCHAR(20),

```

```
Direccion VARCHAR(200),  
  
Contacto VARCHAR(100),  
  
Email VARCHAR(100)  
  
);  
  
GO
```

-- Tabla de Productos

```
CREATE TABLE Productos (  
    IdProducto INT PRIMARY KEY IDENTITY(1,1),  
    Codigo VARCHAR(50) NOT NULL UNIQUE,  
    Nombre VARCHAR(100) NOT NULL,  
    Precio DECIMAL(10,2) NOT NULL,  
    Stock INT NOT NULL DEFAULT 0,  
    IdCategoria INT,  
    IdProveedor INT,  
    FOREIGN KEY (IdCategoria) REFERENCES Categorias(IdCategoria),  
    FOREIGN KEY (IdProveedor) REFERENCES Proveedores(IdProveedor)  
);  
  
GO
```

-- Tabla de Movimientos

```
CREATE TABLE Movimientos (  
    IdMovimiento INT PRIMARY KEY IDENTITY(1,1),  
    Tipo VARCHAR(10) NOT NULL CHECK (Tipo IN ('ENTRADA', 'SALIDA')),  
    Cantidad INT NOT NULL,  
    Fecha DATETIME DEFAULT GETDATE(),
```

```
IdProducto INT NOT NULL,  
FOREIGN KEY (IdProducto) REFERENCES Productos(IdProducto)  
);  
GO
```

6. IMPLEMENTACIÓN (Clases y Lógica)

6.1 Clase DatabaseHelper

Encargada de la conexión a la base de datos.

csharp

```
using System.Data.SqlClient;
```

```
namespace Almacen_EL_Dorado.backend
```

```
{
```

```
public static class DatabaseHelper
```

```
{
```

```
public static string ConnectionString =
```

```
"Server=localhost;Database=AlmacenElDorado;Integrated Security=True;";
```

```
}
```

```
}
```

6.2 Estructura de un Formulario (Ejemplo FrmProductos)

Elementos del formulario:

- txtCodigo: Código del producto
- txtNombre: Nombre del producto

- txtPrecio: Precio del producto
- txtStock: Cantidad en inventario
- cmbCategoria: Selección de categoría
- dgvProductos: Tabla para mostrar productos
- btnNuevo, btnGuardar, btnEditar, btnEliminar, btnBuscar, btnLimpiar

Método para cargar productos:

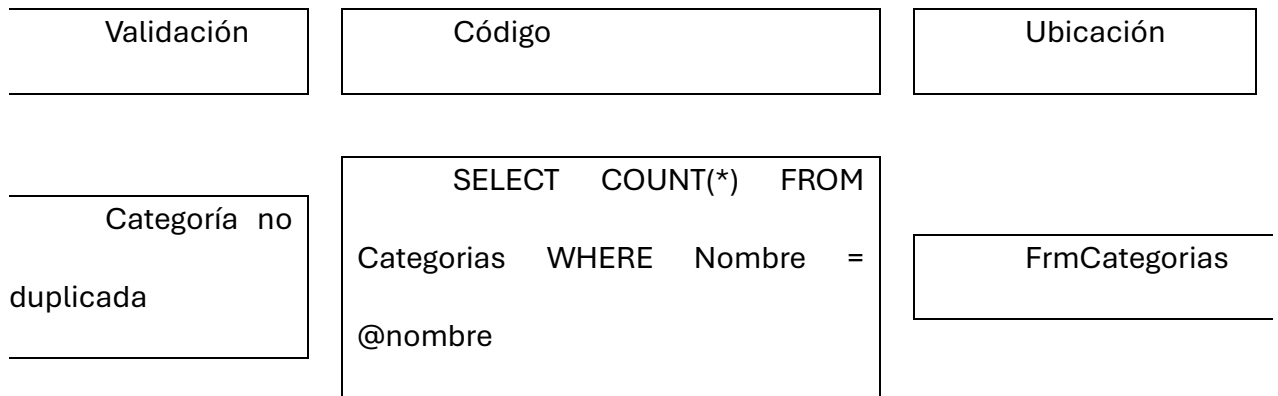
```

csharp
private void CargarProductos(string filtro = "")
{
    using (SqlConnection conn = new
SqlConnection(DatabaseHelper.ConnectionString))
    {
        conn.Open();
        string query = "SELECT p.IdProducto, p.Codigo, p.Nombre, p.Precio, p.Stock,
c.Nombre as Categoria
FROM Productos p
INNER JOIN Categorías c ON p.IdCategoria = c.IdCategoria
ORDER BY p.Nombre";
        SqlCommand cmd = new SqlCommand(query, conn);
        SqlDataReader reader = cmd.ExecuteReader();
        // Llenar DataGridView
    }
}

```

6.3 Validaciones Implementadas

Validación	Código	Ubicación
Campos obligatorios	<code>if (txtCodigo.Text == "")</code>	Todos los formularios
Precio válido	<code>decimal.TryParse()</code>	FrmProductos
Stock suficiente	<code>if (producto.Stock < cantidad)</code>	FrmMovimientos
Código único	<code>SELECT COUNT(*) FROM Productos WHERE Codigo = @codigo</code>	FrmProductos



6.4 Operaciones CRUD

Create (Insertar):

csharp

```
string query = "INSERT INTO Productos (Codigo, Nombre, Precio, Stock,
IdCategoria)
```

```
VALUES (@codigo, @nombre, @precio, @stock, @idCategoria);
```

Read (Leer):

csharp

```
string query = "SELECT * FROM Productos ORDER BY Nombre";
```

Update (Actualizar):

csharp

```
string query = "UPDATE Productos SET Nombre=@nombre, Precio=@precio,
Stock=@stock
```

```
WHERE IdProducto=@id";
```

Delete (Eliminar):

csharp

```
string query = "DELETE FROM Productos WHERE IdProducto=@id";
```

7. RESULTADOS OBTENIDOS

7.1 Funcionalidades Implementadas

Módulo	Funcionalidades	Estado
Login	Autenticación de usuario	 Completo
Productos	CRUD completo, búsqueda, validaciones	 Completo

Módulo	Funcionalidades	Estado
Categorías	Agregar, eliminar, listar	 Completo
Proveedores	CRUD completo, búsqueda por nombre/ID	 Completo
Movimientos	Entradas, salidas, validación de stock	 Completo
Consultas	Búsqueda por nombre, código o categoría	 Completo

7.2 Validación de Stock (Regla de Negocio Crítica)

La validación más importante del sistema es la que impide registrar una salida cuando no hay suficiente stock:

```
csharp
```

```
if (rbSalida.Checked && producto.Stock < cantidad)
```

```
{
```

```
    MessageBox.Show("No hay suficiente stock", "Stock insuficiente");
```

```
    return false;
```

}

8. CONCLUSIONES

8.1 Uso de la Programación Orientada a Objetos

Se aplicaron conceptos fundamentales de POO como:

- **Clases y objetos:** Cada formulario es una clase que se instancia para su uso
- **Encapsulamiento:** Variables privadas y métodos públicos para controlar el acceso

- **Métodos y propiedades:** Organización del código en métodos específicos para cada acción

La POO permitió organizar el código de forma modular, facilitando su mantenimiento y comprensión.

8.2 Experiencia con Bases de Datos

La implementación de SQL Server permitió:

- Almacenar información de forma persistente
- Establecer relaciones entre entidades mediante llaves foráneas
- Realizar consultas complejas con JOIN entre tablas
- Asegurar la integridad de los datos mediante restricciones (CHECK, NOT NULL)

El uso de [ADO.NET](#) con SqlConnection, SqlCommand y SqlDataReader fue fundamental para la interacción entre C# y la base de datos.

8.3 Retos Encontrados

Durante el desarrollo del proyecto se presentaron los siguientes desafíos:

1. **Validación de stock:** Implementar la lógica que impide salidas mayores al stock disponible fue un reto importante, pero se resolvió verificando el stock antes de cada salida.

2. **Manejo de transacciones:** En los movimientos, se implementaron transacciones para asegurar que tanto la actualización del stock como el registro del movimiento se completen juntos o no se realicen.

3. **Organización del código:** Mantener un código limpio y organizado con comentarios claros para facilitar su comprensión.

4. **Integridad referencial:** Manejar las eliminaciones de registros que tienen dependencias (ej: no eliminar una categoría que tiene productos asociados).

8.4 Posibles Mejoras Futuras

El sistema puede ser mejorado con las siguientes funcionalidades:

1. **Reportes en PDF:** Generar reportes de inventario y movimientos en formato PDF

2. **Exportar a Excel:** Permitir exportar las consultas a archivos Excel

3. **Gráficos estadísticos:** Mostrar gráficas de movimientos y stock por categoría

4. **Múltiples usuarios:** Implementar tabla de usuarios con diferentes niveles de acceso

5. **Respaldo automático:** Respaldar la base de datos automáticamente

6. **Código de barras:** Lectura de código de barras para agilizar entradas/salidas

7. **Notificaciones:** Alertas cuando el stock de un producto está bajo

9. BIBLIOGRAFÍA

- Microsoft Docs. (2024). *Windows Forms Documentation*. <https://docs.microsoft.com/en-us/dotnet/desktop/winforms/>

- Microsoft Docs. (2024). *ADO.NET Documentation*. <https://docs.microsoft.com/en-us/dotnet/framework/data/adonet/>

- Microsoft Docs. (2024). *SQL Server*

Documentation. <https://docs.microsoft.com/en-us/sql/>

- Deitel, P., & Deitel, H. (2017). *C# 6 for Programmers*. Pearson.

10. ANEXOS

Anexo A: Enlace al Repositorio GitHub

https://github.com/eramirezt18-hub/Almacen_EL_Dorado.git

Anexo B: Credenciales de Acceso

Campo	Valor
Usuario	AdminSistema
Contraseña	Inventario2024

Anexo C: Requisitos del Sistema

- Sistema Operativo: Windows 7 o superior
- SQL Server Express 2012 o superior
- .NET Framework 4.8
- Visual Studio 2019 o superior (para desarrollo)